

Tuning and Profiling LAMMPS on Deucalion

Humberto Gomes
pg59806@alunos.uminho.pt
University of Minho
Braga, Portugal

José Lopes
pg59807@alunos.uminho.pt
University of Minho
Braga, Portugal

Abstract

Molecular dynamics simulations with LAMMPS can be computationally demanding, requiring efficient use of modern High-Performance Computing (HPC) resources. This paper presents a performance study and tuning of LAMMPS on the ARM partition of the Deucalion supercomputer. We systematically assess the impact of parallelism type, core count, checkpointing strategy, and interconnect selection using the LJ and Rhodo benchmarks. Our results demonstrate that a NUMA-aware MPI+OpenMP configuration achieves a 10.8% performance improvement over a naïve MPI-only approach. Profiling reveals additional optimization opportunities, such as compiler selection and MPI collective directives.

1 Introduction

Molecular dynamics (MD) simulations enable the study of material properties, biological processes, and chemical reactions at the atomic scale. LAMMPS [21] is a widely used, open-source code for performing such simulations. As simulations grow larger, requiring larger systems and longer timescales, efficiently leveraging modern HPC resources becomes critical. Although these systems offer substantial performance potential, realizing it usually requires careful application tuning.

This paper presents a performance study of LAMMPS on the ARM partition of the Deucalion supercomputer [2]. We systematically tune various parameters on two benchmarks with different computational profiles: the compute-bound LJ and the communication-intensive Rhodo, scaling from 1 to 32 nodes. We then profile LAMMPS to identify further potential for optimization.

Our results show that a NUMA-aware MPI+OpenMP configuration yields the best performance. Profiling identified potential future optimizations, such as the use of alternative compilers and collective MPI directives.

2 Background

2.1 LAMMPS

LAMMPS (Large-scale Atomic/Molecular Massively Parallel Simulator) is an open-source classical MD code, widely used for particle-based modeling of materials across various length scales, from atomic and molecular to mesoscale and continuum levels [21].

At its core, LAMMPS integrates Newton’s equations of motion for a system of interacting particles. The forces

between particles are derived from an interatomic potential, of which LAMMPS supports a wide variety, ranging from inexpensive pairwise potentials, such as Lennard-Jones, to computationally demanding manybody and machine-learning potentials. Long-range Coulombic interactions are handled using FFT-based methods such as Particle-Particle Particle-Mesh (PPPM).

For parallel execution, LAMMPS employs a spatial decomposition strategy: the simulation domain is partitioned into non-overlapping subdomains, each assigned to an MPI rank. Each rank stores its own particles, as well as a halo of ghost particles of neighboring subdomains within the short-range interaction cutoff distance. LAMMPS supports diverse domain decomposition strategies, from simple uniform partitioning to complex adaptive strategies for improved load balancing on non-uniform simulations, as shown in Figure 1.

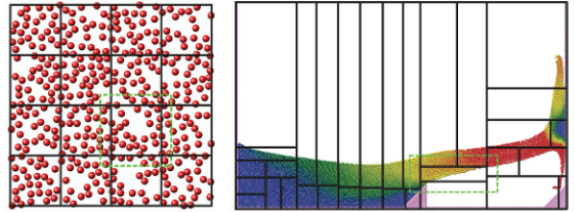


Figure 1. LAMMPS orthogonal spatial decomposition – uniform (left) and non-uniform (right) [21].

Each timestep involves advancing the positions of the particles, communicating ghost particle data with neighboring ranks, rebuilding Verlet-style neighbor lists (if necessary), computing short-range forces and, optionally, using FFT-based solvers for computing long-range interactions. Beyond MPI, LAMMPS supports hybrid parallelism through accelerator packages that leverage thread-level or Single-Instruction Multiple-Data (SIMD) parallelism for performance-critical kernels [5].

2.2 Deucalion

We conducted our work on the Deucalion supercomputer, a heterogeneous cluster comprising multiple partitions with distinct node types [2]. Our experiments used the ARM partition, as it usually offers higher availability, facilitating the allocation of large node counts.

Each ARM node is equipped with a Fujitsu A64FX CPU, an ARMv8.2-A processor with Scalable Vector Extensions (SVE) support. Designed for HPC, the A64FX

architecture, illustrated in Figure 2, comprises 48 cores at 2.0 GHz, organized into four Core Memory Groups (CMGs), with 12 cores per CMG. Memory access is non-uniform (NUMA), with each CMG connected to 8 GiB of High Bandwidth Memory (HBM), totaling 32 GiB per node [2, 19].

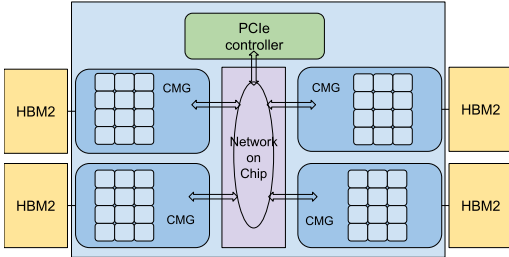


Figure 2. Architecture of the Fujitsu A64FX variant present on Deucalion [13].

Nodes within the same partition are interconnected via a 100 Gb/s InfiniBand fat-tree network for low-latency, high-bandwidth communication, as well as a 1 Gb/s Ethernet network, mainly for management purposes [2].

For storage, each node includes a Western Digital CL SN720 512 GB NVMe SSD. Additionally, all nodes of all partitions are connected to a LustreFS Parallel File System (PFS) [1], which employs a tiered storage architecture combining NVMe SSDs and HDDs. The PFS configuration includes 8 metadata nodes and 32 data nodes, delivering aggregate performance of 340 GB/s for reads and 260 GB/s for writes [2].

3 Benchmarks

For benchmarking LAMMPS, we used two benchmarks with distinct computational characteristics [6]:

- **LJ:** Simulates an atomic fluid, computing only short-range interactions using the Lennard-Jones potential with a 2.5σ cutoff. The absence of long-range interactions means the compute-bound pairwise force calculation dominates this benchmark.
- **Rhodo:** Simulates the rhodopsin protein in a solvated lipid bilayer, using the CHARMM force field with a $10\text{ \AA}$ cutoff for short-range interactions. Long-range Coulombic forces are handled via the PPPM method across compute nodes, making this benchmark more sensitive to communication performance.

During our preliminary analysis, we found that both benchmarks completed in under one minute on a single A64FX core, which was too short for meaningful multi-node scaling studies. To address this, we scaled the simulation axes to increase the number of atoms, thereby extending the runtime. However, large axis scales

led to out-of-memory (OoM) errors, preventing single-node execution and thus full strong scaling analyses. Consequently, we selected the maximum supported axis scale and increased the number of iterations to further extend the runtimes to approximately 30 minutes on a single node. The final benchmark configurations are presented in Table 1.

Table 1. Final configuration of LAMMPS benchmarks.

Benchmark	Axes Scale	Atoms	Iterations
LJ	13	70.3 M	500
Rhodo	5	4 M	500

All tests were conducted on the ARM partition of the Deucalion supercomputer, using LAMMPS 2024-08-29, as provided by Deucalion’s environment modules. This version was compiled using GCC 13.3.0 [20], and uses OpenMPI 5.0.3 [12] for inter-process communication.

4 Tuning

4.1 Methodology

To maximize LAMMPS’ performance, we employed a cascading optimization approach, adjusting one variable at a time and selecting the best-performing configuration before proceeding to the next. While this method significantly reduces the number of required measurements, it may converge only to a local, rather than global, performance optimum.

For each variable, we conducted a strong scaling analysis for both benchmarks to assess LAMMPS’ performance across different node counts. For each node count, we recorded LAMMPS’ self-reported throughput, as well as its per-task runtime breakdown, which we then used to analyze the results. We performed only one run per node count, acknowledging that this approach is susceptible to outlier measurements. However, this was necessary to stay within our allotted compute hours on Deucalion. We remain confident in the reliability of our results, as preliminary testing showed high consistency: the maximum deviation across repeated measurements for the same configuration was just 1.9%.

We tuned the following variables in the specified order:

- **Parallelism type:** On CPUs, LAMMPS supports distributed-memory parallelism via MPI, as well as hybrid parallelism, by combining MPI with either OpenMP or Kokkos [5]. Our goal was to determine whether any parallelism type offers a performance advantage, and whether hybrid parallelism could benefit from NUMA awareness, for example, by running one MPI rank per NUMA node instead of per compute node.

- **Core count:** As a compute-bound application, LAMMPS typically scales efficiently and can leverage high core counts. However, system services and OpenMPI background threads may delay the scheduling of LAMMPS’ threads, potentially degrading overall performance. We hypothesize that reserving a few threads for these tasks may improve performance.
- **Checkpointing strategy:** We introduced periodic checkpointing to disk in the LJ and Rhodo benchmarks to evaluate its impact on application performance. We also explored whether this impact can be mitigated by sharding checkpoint data across multiple files (e.g., one file per MPI rank) or through more complex schemes, such as tiering node-local storage and the PFS [10, 16].
- **Interconnect:** For pedagogical reasons, we assessed the importance of high-performance interconnects in HPC clusters by comparing LAMMPS’ performance using Infiniband, IP over Infiniband (IPoIB) and Ethernet.

4.2 Parallelism Type

To compare distributed-memory and hybrid parallelism, we evaluated the following configurations, each using the maximum number of threads / processes supported, matching the number of cores per node:

- **MPI:** Each compute node runs multiple MPI ranks, one per core, up to 48 processes per node;
- **OMP:** Each compute node runs a single MPI rank, which spawns up to 48 OpenMP threads;
- **OMP-NUMA:** Each compute node runs four MPI ranks, one for each NUMA node, with each spawning up to 12 OpenMP threads;
- **KOKKOS and KOKKOS-NUMA:** Similar to **OMP** and **OMP-NUMA**, respectively, but using Kokkos instead of OpenMP for thread parallelism [22]. These configurations are incompatible with the Rhodo benchmark.

Our results are presented in Figure 3. For the LJ benchmark, all configurations except KOKKOS exhibit linear scaling, with performance differences between them arising only from variations in single-node performance. In contrast, Rhodo, which is more dependent on inter-process communication, does not scale linearly. For example, at 32 nodes using the **OMP-NUMA** configuration, each MPI rank spends on average 29.14% of its time in communication tasks (e.g., synchronizing atoms across ranks or computing long-range interactions), a value which increases with the node count. For LJ, the time in communication-bound tasks is much lower, only 13.6%.

Across both benchmarks and all node counts, **OMP-NUMA** consistently delivers the best performance. At 32 nodes

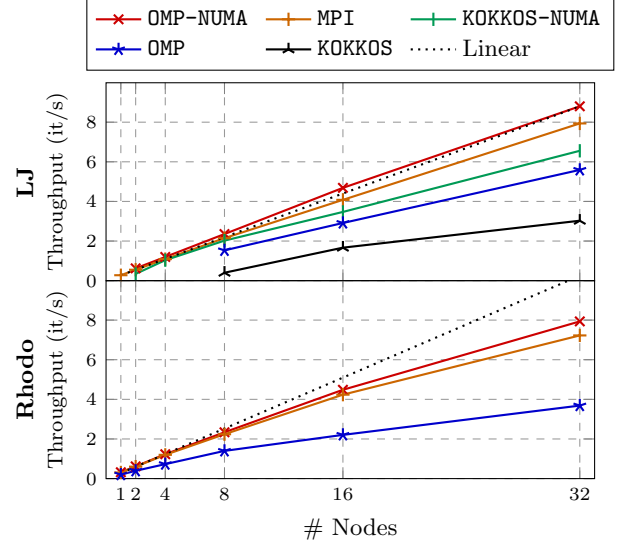


Figure 3. Strong scaling of the LJ and Rhodo benchmarks using different parallelism configurations.

in the LJ benchmark, it yields a 10.8% performance advantage over a naïve MPI configuration and a 190.8% improvement over the worst-performing configuration (KOKKOS). Since LAMMPS parallelism backends are implemented differently from each other, it is impossible to isolate a single factor responsible for the observed performance differences. However, LAMMPS per-task timing breakdowns provide a partial justification: **OMP-NUMA** requires the least time to compute short-range interactions, the most time-consuming task in both benchmarks.

Note that some datapoints for certain parallelism configurations at lower node counts are missing. These configurations resulted in OoM errors, as OpenMP and Kokkos may duplicate data to ensure thread-safe operation [15]. Thus, while some configurations might offer better performance, they may be infeasible due to memory constraints.

4.3 Core Count

Next, we ran LAMMPS using 44 threads per compute node (11 per NUMA node) to investigate whether leaving unallocated cores for system tasks and OpenMPI background threads could improve performance. The results are presented in Figure 4.

For both benchmarks, performance was generally worse with the lower thread count. However, the performance gap between the configurations narrowed as the node count increased, with the 44-thread configuration even outperforming the 48-thread one for Rhodo on 32 nodes.

An analysis of LAMMPS’ per-task time breakdown revealed that this was due to reduced time spent on communication tasks. While close-range force calculations

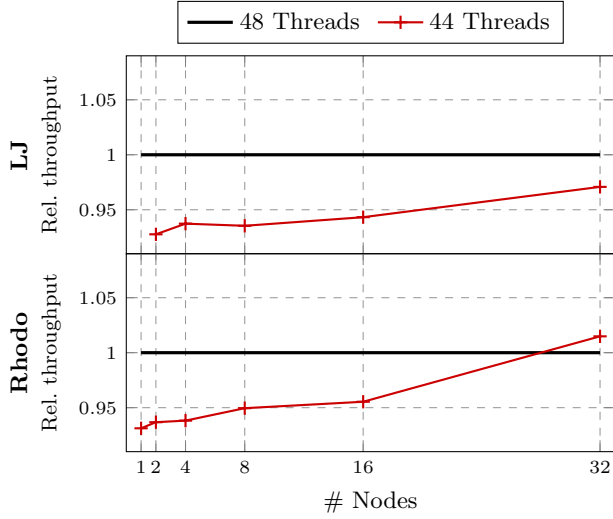


Figure 4. LJ and Rhodos throughput with 44 threads relative to 48 threads, across different node counts.

were faster with 48 threads (due to greater arithmetic capacity), communication, where the 44-thread configuration excelled, became a more significant portion of both benchmarks’ total wall time as the node count increased. Consequently, the performance gap between the two configurations diminished.

Given the overall superior performance of the 48-thread configuration, we used this setup for all subsequent tests. Nevertheless, for node counts exceeding 32, it may be worthwhile to re-evaluate lower thread-count configurations.

4.4 Checkpointing Strategy

Checkpointing — periodically saving the simulation state to persistent storage — is essential for fault tolerance, as it allows for progress recovery in the event of a failure. In our benchmarks, we implemented full simulation checkpointing every 100 iterations using LAMMPS’ `dump` command [7]. At higher node counts, this frequency results in a checkpoint being created every few seconds. While this rate is excessive, it generates an intensive PFS workload, making it easier to distinguish performance differences between checkpointing configurations, of which we tested the following:

- **NOSTORAGE:** No checkpointing is performed;
- **LOCAL:** Each MPI rank writes a checkpoint file to its node’s local NVMe storage. This scenario represents the best-case performance for applications that store checkpoints locally before asynchronously copying them to the PFS [10, 16];
- **PFS-SHARDED:** Each MPI rank writes a checkpoint file directly to the PFS;
- **PFS-SINGLE:** All MPI ranks write to a shared checkpoint file on the PFS.

Our results are presented in Figure 5.

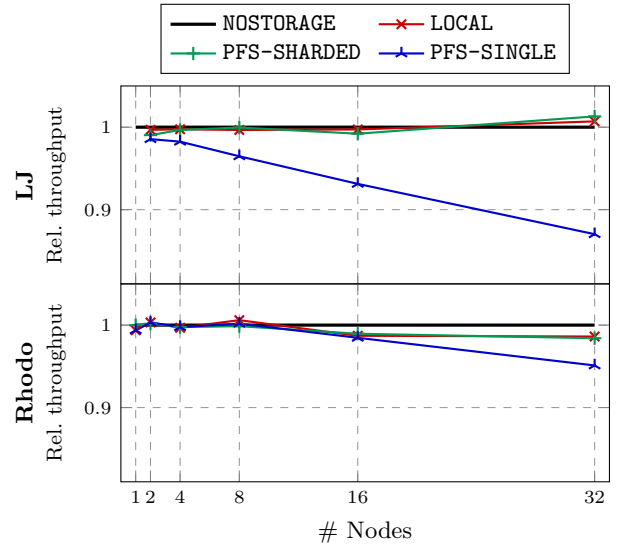


Figure 5. LJ and Rhodos throughput across different node counts and checkpointing techniques (relative to no checkpointing).

We observe that **LOCAL** and **PFS-SHARDED** checkpointing introduce little to no performance degradation. In contrast, **PFS-SINGLE** exhibits a distinct behavior: in the LJ benchmark with 32 nodes, checkpointing accounts for, on average, 6.35 % of rank execution time, compared to only 0.18 % for **PFS-SHARDED**. Straggler ranks, as evidenced by a very high standard deviation in I/O time (119.6 %), further delay checkpointing, resulting in a 13.0 % throughput reduction for **PFS-SINGLE** in relation to **NOSTORAGE**. The effect is less pronounced in Rhodo due to the smaller checkpoint size (914 MiB *vs.* 16 GiB). For subsequent tests, checkpointing will remain disabled.

4.5 Interconnect

Finally, we compared the performance of LAMMPS using Infiniband, IPoIB, and Ethernet. Theoretical bandwidth and latency measurements, obtained via the OSU MPI benchmarks [17], are presented in Table 2. These results reveal differences of up to 104× in bandwidth and 17× in latency among the interconnects.

Table 2. Unidirectional bandwidth and ping-pong latency of the interconnects tested.

Interconnect	Bandwidth (MB/s)	Latency (μ s)
Infiniband	12 298.6	2.85
IPoIB	1 515.0	39.17
Ethernet	117.5	48.55

We then evaluated how these theoretical differences affect LAMMPS performance. Our results are presented in Figure 6.

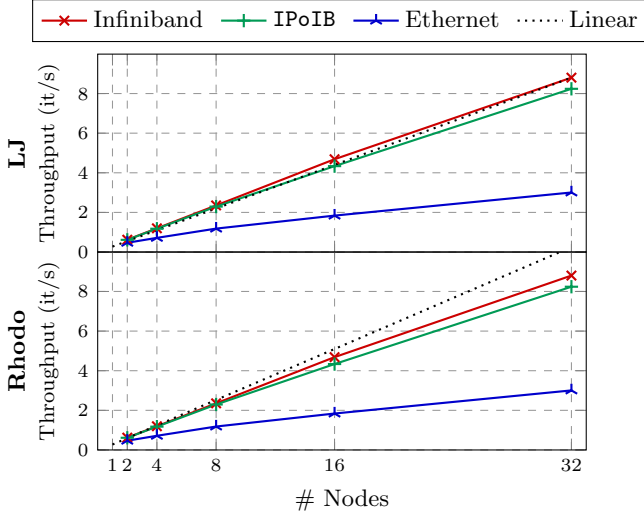


Figure 6. Strong scaling of the LJ and Rhodo benchmarks using different interconnects.

Our analysis shows negligible performance differences between Infiniband and IPoIB. However, when using Ethernet, both benchmarks become communication-bound, resulting in poor scalability. For 32-node runs, LAMMPS’ runtime breakdowns indicate that, compared to Infiniband, a node running the LJ benchmark spends, on average, 70.37 % of its time on communication tasks (*vs.* 13.61 %), while the Rhodo benchmark spends, on average, 75.35 % (*vs.* 29.14 %). This leads to performance degradations of 65.6 % (LJ) and 77.4 % (Rhodo) at 32 nodes. This phenomenon is explored in greater detail in Section 5.5.

In summary, optimal LAMMPS performance is achieved using the OMP-NUMA parallelism configuration, with 48 LAMMPS threads per compute node and the Infiniband interconnect. Future tests will be performed without checkpointing, but if properly configured (e.g., by using sharding), checkpointing does not harm performance at the scale and frequency tested.

5 Profiling

5.1 Methodology

Following tuning, we profiled LAMMPS to identify the root causes of the previous results and to explore potential configuration adjustments for further optimization. Our profiling focused on the following questions:

- Could performance be further improved by replacing libraries (e.g., BLAS) with A64FX-optimized versions?

- What communication patterns dominate LAMMPS, and how do they scale with node count? Is there evidence of load imbalance?
- How does MPI communication performance compare between Ethernet and Infiniband?

To answer these questions, we profiled three configurations of the Rhodo benchmark: 1-node and 32-node setups in their optimal configurations, and a 32-node setup with Ethernet instead of Infiniband. We used NVIDIA NSight Systems 25.1a [3] to trace MPI calls and sample regular function calls.

5.2 Profiling Overhead

During preliminary testing, we observed significant profiling overhead, as shown in Figure 7.

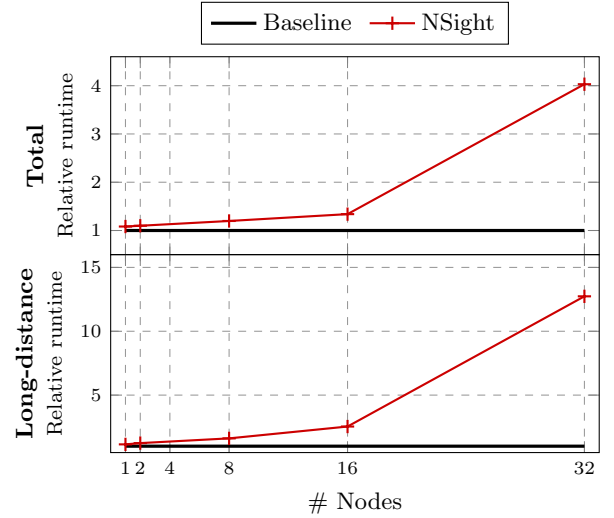


Figure 7. Runtimes of the Rhodo benchmark (total and long-range interaction calculations) relative to their non-profiled equivalents, across different node counts.

The minimum profiling overhead observed was 8.2 % for a single node. However, for 32 nodes, profiling increased LAMMPS’ self-reported execution time fourfold! Analysis of LAMMPS’ per-task runtime breakdowns revealed that most of the overhead originated from the communication-bound long-range force calculations, which, for 32 nodes, were $12.7\times$ slower when profiled.

To mitigate this overhead, we tested the following approaches in isolation: disabling MPI tracing, disabling function call sampling, and profiling only one MPI rank per compute node. However, these adjustments yielded negligible improvements in an 8-node configuration. Ideally, we would have adopted lightweight profilers such as mpiP for MPI tracing [14] and perf record [18] for function call sampling, but time constraints prevented their

implementation. Therefore, we replaced our 32-node tests with 8-node tests, limiting overhead to 19.7%.

5.3 Further Optimization Potential

To identify kernels with potential for further optimization, we analyzed the call graph of the first MPI rank in a single-node run. The dominant hotspot is the `PairLJCharmmCoulLongOMP::compute` kernel, which consumes 62.51% of CPU time. This kernel, responsible for computing short-range forces at each timestep, is highly compute-bound. Thus, it could benefit from optimizations such as more efficient code generation using alternative compilers [8], or a manual rewrite to leverage ARM SVE instructions for SIMD processing. For comparison, the INTEL package enables SIMD processing on x86 processors and has demonstrated 2× speedups in previous work [15].

While no other kernel emerged as an immediate candidate for optimization, we also evaluated external libraries for potential replacement with more efficient alternatives. For instance, FFTW [9], used in long-range force computations, could be substituted with Fujitsu’s A64FX-tuned FFTW implementation [11]. However, the expected performance gains would be minimal, as the FFTW kernel accounts for only 0.44% of total CPU usage in the profiled configuration.

5.4 MPI Communication Breakdown

Next, we analyzed the MPI communication patterns of LAMMPS, comparing single-node and 8-node configurations. Tables 3 and 4 report the time share of each of each MPI directive in relation to total communication time and overall application wall time.

Table 3. MPI directive time on the 1-node configuration.

MPI Directive	MPI Time (%)	Wall Time (%)
<code>MPI_Wait</code>	41.0	1.32
<code>MPI_Allreduce</code>	31.0	1.01
<code>MPI_Send</code>	21.0	0.68
Others	7.0	–

Table 4. MPI directive time on the 8-node configuration.

MPI Directive	MPI Time (%)	Wall Time (%)
<code>MPI_Wait</code>	41.0	7.69
<code>MPI_Allreduce</code>	28.0	5.30
<code>MPI_Send</code>	16.0	2.98
<code>MPI_Waitany</code>	7.0	1.45
Others	8.0	–

The distribution of communication time across MPI primitives is consistent between configurations: both are dominated by time spent waiting for messages and performing all-reductions. However, with 8 nodes, waiting becomes even more pronounced, with the `MPI_Waitany` directive also contributing significantly. Despite the similar breakdown of communication types, the overall communication time increases substantially with node count, accounting for 17.87% of wall time in the 8-node configuration compared to 3.08% in the single-node case. This trend is expected, as multi-node setups require additional synchronization and, unlike the single-node case, use Infiniband rather than shared memory.

Additionally, we observed evidence of load imbalance, which arises because different MPI ranks process varying numbers of atoms. A profiler timeline analysis revealed that straggler nodes delayed progress at synchronization barriers. LAMMPS’ internal instrumentation further confirmed this, showing performance disparities of up to 25.1% between the slowest and fastest MPI ranks in the 8-node configuration. There are also indications of load imbalance among OpenMP threads, with up to 5.9% of CPU time spent in OpenMP spinlock barriers. However, this metric may not accurately reflect the true imbalance, as cores can idle in these barriers, halting sampling. Unfortunately, we were unable to configure the profiler to correctly trace OpenMP events.

One potential optimization to reduce communication overhead is to replace node-to-node messaging with collective MPI directives in the PPPM long-range interaction calculation [4].

5.5 Interconnect

Finally, we evaluated the impact of replacing Infiniband with Ethernet in an 8-node configuration. The resulting MPI directive breakdown is presented in Table 5. As expected, performance degraded significantly: every directive accounts for a substantially larger fraction of wall time compared to the Infiniband configuration (Table 4), and MPI communication now consumes 75.7% of the total runtime.

Table 5. MPI directive time on a 8-node Ethernet configuration.

MPI Directive	MPI Time (%)	Wall Time (%)
<code>MPI_Send</code>	58.0	44.70
<code>MPI_Waitany</code>	25.0	19.60
<code>MPI_Wait</code>	12.0	9.21
<code>MPI_Allreduce</code>	2.0	2.19
Others	3.0	–

The most notable change is the dramatic increase in MPI.Send time, which now dominates the runtime. This shift indicates a bandwidth-limited configuration, as Ethernet’s bandwidth is 104× lower than that of Infiniband. These results highlight the critical role of high-performance interconnects in HPC clusters.

6 Conclusion

In this paper, we optimized LAMMPS’ configuration for best performance on Deucalion’s ARM partition. By employing hybrid MPI+OpenMP NUMA-aware parallelism, we achieved performance improvements of up to 10.8% over a naïve MPI configuration. We also demonstrated that sharding can eliminate the performance overhead of simulation checkpointing, and that interconnect performance can become a bottleneck, even for compute-bound applications like LAMMPS. Finally, through profiling, we identified potential optimizations for future work, including enabling collective MPI directives, exploring alternative compilers, or extending LAMMPS to support SIMD parallelism on the A64FX via SVE instructions.

Acknowledgments

This work made use of compute hours awarded by grant 2025.00010.HPCVLAB.UMINHO, funded by the Foundation for Science and Technology (FCT).

References

- [1] Peter Braam. 2019. *The Lustre Storage Architecture*. arXiv:1903.01955
- [2] CNCA. 2026. *Deucalion*. Retrieved Jun 8, 2026 from <https://docs.macc.fcn.pt/deucalion/>
- [3] NVIDIA Corporation. 2026. *NVIDIA Nsight Systems*. Retrieved Jun 9, 2026 from <https://developer.nvidia.com/nsight-systems>
- [4] Sandia Corporation. 2026. *kspace_modify command*. Retrieved Jun 11, 2026 from https://docs.lammps.org/kspace_modify.html
- [5] Sandia Corporation. 2026. *LAMMPS – Accelerator Packages*. Retrieved Jun 9, 2026 from https://docs.lammps.org/Speed_packages.html
- [6] Sandia Corporation. 2026. *LAMMPS – Benchmarks*. Retrieved Jun 8, 2026 from https://docs.lammps.org/Speed_bench.html
- [7] Sandia Corporation. 2026. *LAMMPS – dump*. Retrieved Jun 10, 2026 from <https://docs.lammps.org/dump.html>
- [8] James Fischer, Vincent Natoli, and David Richie. 2006. Optimization of LAMMPS. In *2006 HPCMP Users Group Conference (HPCMP-UGC’06)*. 374–377. doi:10.1109/HPCMP-UGC.2006.56
- [9] Matteo Frigo and Steven Johnson. 2005. The Design and Implementation of FFTW3. *Proc. IEEE* 93, 2 (2005), 216–231. doi:10.1109/JPROC.2004.840301
- [10] Nicholas Frontiere, J. Emberson, Michael Buehlmann, Esteban Rangel, Salman Habib, Katrin Heitmann, Patricia Larsen, Vitali Morozov, Adrian Pope, Claude-André Faucher-Giguère, Antigoni Georgiadou, Damien Lebrun-Grandié, and Andrey Prokopenko. 2025. Cosmological Hydrodynamics at Exascale: A Trillion-Particle Leap in Capability. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. Association for Computing Machinery, 25–35. doi:10.1145/3712285.3771786
- [11] Fujitsu. 2024. *FFTW3*. Retrieved June 11, 2026 from <https://github.com/fujitsu/fftw3>
- [12] Richard Graham, Timothy Woodall, and Jeffrey Squyres. 2006. Open MPI: A Flexible High Performance MPI. In *Parallel Processing and Applied Mathematics*. 228–239. doi:10.1007/11752578_29
- [13] Tomoya Kusaba, Yusuke Awaki, Kohei Yoshida, Shinobu Miwa, Hayato Yamaki, Toshihiro Hanawa, and Hiroki Honda. 2024. Power-Efficiency Variation on A64FX Supercomputers and its Application to System Operation. In *2024 IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*. 55–65. doi:10.1109/CLUSTERWorkshops61563.2024.00018
- [14] Lawrence Livermore National Laboratory. 2026. *NVIDIA Corporation – A Lightweight MPI Profiler*. Retrieved Jun 9, 2026 from <https://software.llnl.gov/mPiP/>
- [15] Stan Moore. 2017. *Accelerating LAMMPS Performance*. Retrieved Jun 10, 2026 from <https://www.lammps.org/workshops/Aug17/pdf/moore.pdf>
- [16] Bogdan Nicolae, Jiali Li, Justin Wozniak, George Bosilca, Matthieu Dorier, and Franck Cappello. 2020. DeepFreeze: Towards Scalable Asynchronous Checkpointing of Deep Learning Models. In *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. 172–181. doi:10.1109/CCGrid49817.2020.00-76
- [17] Dhabaleswar Panda. 2025. *OSU Micro-Benchmarks 7.5.2*. Retrieved Jun 9, 2026 from <https://mvapich.cse.ohio-state.edu/benchmarks/>
- [18] Linux perf wiki Contributors. 2024. *perf: Linux Profiling with Performance Counters*. Retrieved Jun 9, 2026 from <https://perfwiki.github.io/>
- [19] Mitsuhsa Sato, Yutaka Ishikawa, Hirofumi Tomita, Yuetsu Kodama, Tetsuya Odajima, Miwako Tsuji, Hisashi Yashiro, Masaki Aoki, Naoyuki Shida, Ikuo Miyoshi, Kouichi Hirai, Atsushi Furuya, Akira Asato, Kuniki Morita, and Toshiyuki Shimizu. 2020. Co-Design for A64FX Manycore Processor and “Fugaku”. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15. doi:10.1109/SC41405.2020.00051
- [20] Richard Stallman and GCC Developer Community. 2023. *Using the GNU Compiler Collection – For GCC Version 13.3.0*. GNU Press, Boston, MA, USA. <https://gcc.gnu.org/onlinedocs/gcc-13.3.0/gcc.pdf>
- [21] Aidan Thompson, Metin Aktulga, Richard Berger, Dan Bolinteanu, Michael Brown, Paul Crozie, Pieter Veld, Axel Kohlmeyer, Stan Moore, Trung Nguyen, Ray Shan, Mark Stevens, Julien Tranchida, Christian Trott, and Steven Plimpton. 2022. LAMMPS – a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications* 271 (2022). doi:10.1016/j.cpc.2021.108171
- [22] Christian Trott, Damien Lebrun-Grandié, Daniel Arndt, Jan Ciesko, Vinh Dang, Nathan Ellingwood, Rahul Kumar Gayatri, Evan Harvey, Daisy Hollman, Dan Ibanez, Nevin Liber, Jonathan Madsen, Jeff Miles, David Poliakoff, Amy Powell, Sivasankaran Rajamanickam, Mikael Simberg, Dan Sunderland, Bruno Turcksin, and Jeremiah Wilke. 2022. Kokkos 3: Programming Model Extensions for the Exascale Era. *IEEE Transactions on Parallel and Distributed Systems* 33, 4 (2022), 805–817. doi:10.1109/TPDS.2021.3097283